

Redux and Firestore Study Notes

Project: Warhammer40KApp

Goal: explain how Firebase Auth, account lifecycle, Firestore, Redux, Redux Persist, slices, reducers, dispatch, selectors, and the related design patterns work in this app.

Important defense wording: if Redux or Firestore was required by the assignment, do not say "I chose it." Say "The requirement was to use it, and I implemented it in this structure."

Table of Contents

1. Firebase Authentication and How It Connects To Firestore
2. Auth and Account Lifecycle
3. Firestore: Collections, Documents, Reads, Writes, Listeners, and Patterns
4. Firestore Data Modeling
5. Firestore Security Rules
6. Real-Time Listener Lifecycle
7. Offline and Local Persistence
8. Redux: Store, Slices, Reducers, Actions, Dispatch, and Selectors
9. Redux Toolkit Versus Classic Redux
10. Immutability and Immer
11. Redux Persist
12. Selectors
13. Optimistic Updates and Failure Handling
14. How Redux and Firestore Work Together In This App
15. Short Defense Answers

1. Firebase Authentication and How It Connects To Firestore

Important distinction

Firebase Authentication and Firestore are related, but they are not the same thing.

- **Firebase Authentication** proves who the user is. It handles register, login, logout, password updates, and email updates.
- **Firestore** stores database documents, such as armies and army compositions.

Authentication answers the question: "Who is using the app?" Firestore answers the question: "What data should be stored or loaded?"

Firebase setup

In `src/config/firebase.ts`, the app creates both an Auth instance and a Firestore instance:

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`auth` is the object used by Firebase Authentication functions. `database` is the object used by Firestore functions.

Why auth is passed as the first parameter

Firebase uses the modular SDK style. In this style, most Firebase functions receive the service instance as their first argument.

For example, in `src/services/authService.ts`:

```
export const login = (auth: Auth, email: string, password: string) => {
  return signInWithEmailAndPassword(auth, email, password);
};
```

The call looks like this:

```
signInWithEmailAndPassword(auth, email, password)
```

The first parameter, `auth`, tells Firebase which Firebase Auth instance should handle this login request. The email and password are the actual credentials being checked.

This is similar to Firestore functions using the database instance:

```
getDocs(collection(database, "armies"))
```

Here, `database` tells Firebase which Firestore database to read from.

Defense answer: Firebase's modular API passes the service instance first. For authentication functions, that first parameter is `auth`. For Firestore functions, it is usually `database` or a document/collection reference created from `database`.

Why the service functions receive `auth` as a parameter

Your service functions are written like this:

```
export const register = (auth: Auth, email: string, password: string) => {  
  return createUserWithEmailAndPassword(auth, email, password);  
};
```

They could have imported `auth` directly inside the service file, but this version passes `auth` in from the screen:

```
await login(auth, email, password);
```

This makes the service function explicit: it needs an `Auth` instance, an email, and a password. It also makes the function easier to test or reuse with another `Auth` instance.

Register flow

When the user registers, the app calls:

```
await register(auth, email, password);
```

The service calls:

```
createUserWithEmailAndPassword(auth, email, password)
```

Firebase creates the account. After registration, Firebase automatically signs in the new user. That login state change is detected by `onAuthStateChanged` in `AuthContext`.

Login flow

When the user logs in, the app calls:

```
await login(auth, email, password);
```

The service calls:

```
signInWithEmailAndPassword(auth, email, password)
```

Firebase checks the email and password. If they are correct, Firebase updates the auth state. Then `AuthContext` receives the logged-in user.

AuthContext watches login state

The important listener is in `src/contexts/AuthContext.tsx`:

```
const unsubscribe = onAuthStateChanged(auth, (user) => {  
  setCurrentUser(user);  
  setLoading(false);  
});
```

`onAuthStateChanged` watches Firebase Auth. It runs when:

- the user logs in
- the user logs out
- the app reloads and Firebase checks whether a saved session still exists

If a user is logged in, `user` is a Firebase `User` object. If no user is logged in, `user` is `null`.

How authentication connects to Firestore

After login, Firebase gives the app a user object. The most important field for Firestore paths is:

```
currentUser.uid
```

The `uid` is Firebase's unique id for that authenticated user. Your app uses it in Firestore paths:

```
collection(database, "users", userId, "armyCompositions")
```

When the hook calls the service, it passes the logged-in user's uid:

```
subscribeToArmyCompositions(  
  database,  
  currentUser.uid,  
  (armyCompositions) => {  
    dispatch(armyCompositionsLoaded(armyCompositions));  
  },  
);
```

This creates a path like:

```
users/{currentUser.uid}/armyCompositions
```

So the logged-in user's id decides which Firestore subcollection is loaded.

Defense answer: Firebase Auth identifies the user. The app then uses `currentUser.uid` to

read and write that user's Firestore data under `users/{uid}` .

Firestore security rules concept

In the app code, using `currentUser.uid` creates the correct path. But real protection should also happen in Firestore security rules.

A typical rule concept is:

```
allow read, write: if request.auth != null
  && request.auth.uid == userId;
```

This means a user may only access documents where the path's `userId` matches their authenticated Firebase uid.

The frontend path is important for organization. Firestore rules are important for security.

Defense answer: The frontend uses the authenticated user's uid to build the correct Firestore path. Firestore security rules should enforce that users can only access their own documents.

Why sensitive account changes reauthenticate

Changing email or password is sensitive. Firebase often requires recent authentication before allowing those operations.

```
const credential = EmailAuthProvider.credential(user.email, password);

await reauthenticateWithCredential(user, credential);
```

This proves that the person using the app still knows the current password. After that, Firebase allows sensitive operations like updating email or password.

```
await verifyBeforeUpdateEmail(user, newEmail);
```

This sends a verification email before changing to the new email address.

```
return updatePassword(user, newPassword);
```

This updates the Firebase Auth password after reauthentication succeeds.

Logout flow

Logout calls:

```
await logout(auth);
```

The service calls:

```
signOut(auth)
```

Firebase signs the user out. Then `onAuthStateChanged` receives `null`, `currentUser` becomes `null`, and `App.tsx` switches back to the auth screens.

Defense summary: Firebase Authentication handles identity. The app passes the `auth` instance to Firebase Auth functions because that is how the modular SDK knows which Auth service to use. Once a user is authenticated, their `uid` is used to load and save their Firestore data in a user-owned path.

2. Auth and Account Lifecycle

What account lifecycle means

The account lifecycle is the full path of a user's identity in the app: register, login, restore saved login, update account details, and logout.

Register/login

- > Firebase Auth creates or verifies identity
- > AuthContext receives user
- > currentUser becomes available
- > currentUser.uid is used for Firestore paths
- > logout sets currentUser back to null

Register lifecycle

The register screen calls the auth service:

```
await register(auth, email, password);
```

The service calls Firebase:

```
createUserWithEmailAndPassword(auth, email, password)
```

Firebase creates the account and signs the new user in. That sign-in triggers `onAuthStateChanged`, which updates `AuthContext`.

Login lifecycle

```
await login(auth, email, password);
```

The service calls:

```
signInWithEmailAndPassword(auth, email, password)
```

Firebase verifies the credentials. If they are correct, Firebase changes the auth state. The app does not need to manually set `currentUser` in the login screen; `AuthContext` receives the user through the listener.

Restoring a saved login

When the app reloads, Firebase checks whether there is still a saved authenticated session. During that check, `loading` is true.

```
const [loading, setLoading] = useState(true);
```

When Firebase finishes checking, the listener runs:

```
const unsubscribe = onAuthStateChanged(auth, (user) => {  
  setCurrentUser(user);  
  setLoading(false);  
});
```

If Firebase finds a saved login, `user` is a Firebase user. If not, `user` is `null`.

Account update lifecycle

Email and password changes are sensitive. Firebase requires recent authentication before allowing them.

```
const credential = EmailAuthProvider.credential(user.email, password);  
  
await reauthenticateWithCredential(user, credential);
```

This proves the current user still knows their password. After reauthentication, the app can request the sensitive update.

```
await verifyBeforeUpdateEmail(user, newEmail);
```

This sends a verification email before changing the email address.

```
return updatePassword(user, newPassword);
```

This updates the user's Firebase Auth password after reauthentication succeeds.

Logout lifecycle

```
dispatch(arrayCompositionsCleared());  
  
await logout(auth);
```

The app clears local Redux army compositions before signing out. Then Firebase signs the user out. `onAuthStateChanged` receives `null`, and `AppContent` switches back to the auth stack.

Defense answer: Firebase Auth owns the account lifecycle. The app registers, logs in, restores sessions, reauthenticates for sensitive updates, and logs out through Firebase Auth. `AuthContext` listens to those changes and exposes `currentUser` to the app.

3. Firestore: Collections, Documents, Reads, Writes, Listeners, and Patterns

What Firestore is

Firestore is a NoSQL cloud database from Firebase. It stores data in collections and documents instead of SQL tables and rows.

Traditional SQL databases use:

```
tables
rows
columns
relations
```

Firestore uses:

```
collections
documents
fields
subcollections
```

Firestore data model

A collection is a group of documents. A document is like a JavaScript object with fields.

```
{
  id: "abc",
  name: "My Army",
  totalPoints: 500
}
```

A subcollection is a collection inside a document.

Firestore in your app

The Firebase setup is in `src/config/firebase.ts`:

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`initializeApp` connects the React Native app to the Firebase project. `auth` is the Firebase Auth service. `database` is the Firestore database connection.

Service layer pattern

Firestore functions are placed in `src/services/firestoreService.ts`.

This is a service layer. Screens do not directly contain Firestore calls. Instead, they call service functions like:

- `subscribeToArmyCompositions`
- `saveArmyComposition`
- `updateArmyCompositionUnits`
- `deleteArmyComposition`
- `getArmies`

Defense answer: Firestore access is isolated in a service file so the screens do not directly depend on Firebase implementation details.

Your Firestore structure

User-created army compositions are stored like this:

```
users/{userId}/armyCompositions/{armyCompositionId}
```

In code, that looks like this:

```
doc(database, "users", userId, "armyCompositions", armyComposition.id)
```

Example structure:

```
users
  alexUserId
    armyCompositions
      army1
      army2

  otherUserId
    armyCompositions
      army3
```

This is a user-owned data pattern. Each authenticated user has their own army compositions subcollection.

Defense answer: The app stores army compositions under the authenticated user's id. That keeps each user's data logically separated and makes security rules easier because access can be checked against the user id.

collection versus doc

This points to a collection:

```
collection(database, "users", userId, "armyCompositions")
```

This points to one specific document:

```
doc(database, "users", userId, "armyCompositions", armyComposition.id)
```

`collection(...)` means "a list of documents." `doc(...)` means "one specific document."

Reading data once with `getDocs`

The app uses `getDocs` to load the available armies:

```
const querySnapshot = await getDocs(collection(database, "armies"));
```

`getDocs` reads data once. In this app, that is used for shared army reference data.

Defense answer: The app uses `getDocs` for armies because armies are shared reference data and do not need constant real-time updates while the user is on the screen.

Listening in real time with `onSnapshot`

The app uses `onSnapshot` for user army compositions:

```
return onSnapshot(
  collection(database, "users", userId, "armyCompositions"),
  (querySnapshot) => {
    const armyCompositions = querySnapshot.docs.map((document) => {
      return document.data() as ArmyComposition;
    });

    onArmyCompositionsChanged(armyCompositions);
  },
);
```

`onSnapshot` keeps listening. When Firestore data changes, Firebase sends a new snapshot to the app.

This is the observer or subscription pattern. The app subscribes to changes, and Firestore notifies the app when data changes.

Defense answer: The app uses Firestore's real-time listener so local state stays synchronized with the database. When Firestore changes, the listener receives a snapshot and dispatches the new data into Redux.

Saving data with `setDoc`

This saves a full army composition:

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyComposition.id),  
  armyComposition,  
);
```

`doc(...)` creates a reference to the document. `setDoc(...)` writes the data to that document. If the document does not exist, Firestore creates it. If it exists, Firestore replaces it unless merge is used.

Updating part of a document with merge

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyCompositionId),  
  {  
    units,  
    totalPoints,  
  },  
  { merge: true },  
);
```

`{ merge: true }` means update only the provided fields. In this case, Firestore updates `units` and `totalPoints`, while keeping fields like `id`, `name`, and `army`.

Defense answer: `merge: true` is used so updating units does not overwrite the whole army composition document.

Deleting data

```
return deleteDoc(  
  doc(database, "users", userId, "armyCompositions", armyCompositionId),  
);
```

`deleteDoc` removes one specific document from Firestore.

Important Firestore patterns in this app

- **User-owned data pattern:** each user's data is stored under `users/{userId}`.
- **Service layer pattern:** Firestore logic lives in service functions, not directly in screens.
- **Observer/subscription pattern:** `onSnapshot` listens for real-time database changes.
- **Optimistic UI pattern:** the app sometimes updates Redux before Firestore finishes saving, so the UI feels faster.
- **Query-shaped data pattern:** Firestore documents are shaped around how the app reads them, because Firestore is not built around SQL joins.

- **Async data pattern:** reads and writes use `async` and `await` because database work takes time.

4. Firestore Data Modeling

Why Firestore data modeling is different from SQL

Firestore is document-based, not table-based. SQL databases usually normalize data into related tables. Firestore usually stores documents in the shape the app needs to read them.

SQL mindset

- > tables
- > rows
- > joins
- > normalized relationships

Firestore mindset

- > collections
- > documents
- > subcollections
- > query-shaped documents

Shared army data

The selectable armies are stored in a top-level collection:

```
armies/{armyId}
```

This data is shared reference data. It is not owned by one specific user.

User-owned army compositions

User-created army compositions are stored under each user's uid:

```
users/{userId}/armyCompositions/{armyCompositionId}
```

This makes ownership clear. If the logged-in user has uid `abc123`, the app reads and writes:

```
users/abc123/armyCompositions
```

Why army compositions store copied army data

An `ArmyComposition` stores the selected `army` object directly:

```
export type ArmyComposition = {  
  id: string;  
  name: string;  
  army: Army;  
  units: SelectedUnit[];
```

```
totalPoints: number;  
};
```

This means the detail screen can display the composition without doing a second Firestore read or a join. That matches Firestore's query-shaped data pattern.

Tradeoff: Copying army data makes reads simple, but if the shared army data changes later, older saved army compositions may still contain the old copied data.

Why selected units need their own id

The same unit type can be added more than once. That is why the app adds `armyCompositionUnitId` to each selected copy. It lets the app remove one specific selected copy instead of every unit with the same original unit id.

Defense answer: Firestore data is modeled around how the app reads it. Shared armies live in a top-level collection, while user-created compositions live under `users/{uid}`. Army data is copied into compositions to make the detail screen simple to render.

5. Firestore Security Rules

Frontend paths are not enough

The app builds the correct user-owned path with `currentUser.uid`, but that alone is not security. A malicious client could try to call Firestore with another path. Firestore security rules must enforce access on the backend.

Basic rule concept

```
allow read, write: if request.auth != null
  && request.auth.uid == userId;
```

This means the request must come from a logged-in user, and the logged-in user's uid must match the `userId` in the Firestore path.

How this matches the app structure

Firestore path:
`users/{userId}/armyCompositions/{armyCompositionId}`

Authenticated Firebase user:
`request.auth.uid`

Security idea:
`request.auth.uid` must equal `userId`

Shared armies rule concept

The `armies` collection is shared reference data. A common rule design is to allow authenticated users to read it, but restrict writes to admins or trusted server-side tooling.

```
allow read: if request.auth != null;
allow write: if false;
```

This is a conceptual example, not necessarily the exact deployed rule. The important defense idea is that shared reference data and user-owned data should have different access rules.

Why client validation is not security

Formik and Yup validate input in the app. That improves user experience, but client-side validation can be bypassed. Security rules should still protect data access and required ownership.

Defense answer: The frontend uses `currentUser.uid` to build user-owned paths, but Firestore security rules should enforce that users can only access their own documents. Client-side code organizes access; backend rules protect it.

6. Real-Time Listener Lifecycle

What a listener lifecycle is

A listener lifecycle is the process of starting a listener, receiving updates, and cleaning it up when it is no longer needed.

Firestore listener

```
return onSnapshot(  
  collection(database, "users", userId, "armyCompositions"),  
  (querySnapshot) => {  
    const armyCompositions = querySnapshot.docs.map((document) => {  
      return document.data() as ArmyComposition;  
    });  
  
    onArmyCompositionsChanged(armyCompositions);  
  },  
);
```

`onSnapshot` starts listening to a collection. It immediately sends data and then sends new snapshots when the collection changes.

Returned unsubscribe function

`onSnapshot` returns a function that stops the listener. The app stores that function as `unsubscribe`.

```
const unsubscribe = subscribeToArmyCompositions(...);  
  
return unsubscribe;
```

Returning `unsubscribe` from `useEffect` gives it to React as the cleanup function.

Why cleanup matters

Cleanup prevents old listeners from staying active forever. This matters when the user logs out, another user logs in, or the component unmounts.

```
currentUser changes  
-> old effect cleanup runs  
-> old Firestore listener unsubscribes  
-> new effect starts listener for new user
```

Auth listener lifecycle

`AuthContext` uses the same pattern:

```
const unsubscribe = onAuthStateChanged(auth, (user) => {  
  setCurrentUser(user);  
  setLoading(false);  
});  
  
return unsubscribe;
```

Firebase Auth also returns an unsubscribe function. React calls it when the provider unmounts.

Defense answer: Firebase listeners follow a subscription pattern. The app starts listeners with `onSnapshot` or `onAuthStateChanged`, receives updates through callbacks, and returns the unsubscribe function from `useEffect` so React can clean them up.

7. Offline and Local Persistence

Two different persistence ideas

This app has two persistence layers to understand:

- **Firestore:** remote cloud persistence for user data.
- **Redux Persist with AsyncStorage:** local device persistence for Redux state.

Firestore persistence

Firestore is the source of remote saved data. When the app saves an army composition, it writes to Firestore so the data belongs to the user's account.

```
await saveArmyComposition(database, currentUser.uid, newArmyComposition);
```

Redux Persist local storage

Redux Persist saves Redux state into AsyncStorage:

```
const persistConfig = {  
  key: "root",  
  storage: AsyncStorage,  
};
```

This helps Redux state survive app restarts locally on the device.

Important distinction

Redux Persist is not the same as Firestore. Redux Persist is local to the device. Firestore is remote account data. If the user installs the app on another device, Firestore is the database that can provide their saved army compositions.

Honest defense: This project explicitly configures Redux Persist with AsyncStorage. It does not explicitly configure custom Firestore offline persistence behavior, so do not defend Firestore offline mode as a custom implemented feature.

Why both can exist

Redux Persist can make the app feel faster locally because Redux can rehydrate cached state. Firestore remains the authoritative remote data source for user-owned army compositions.

Defense answer: Firestore persists user data remotely, while Redux Persist saves Redux state locally with AsyncStorage. They solve different problems: remote account storage versus local app-state restoration.

8. Redux: Store, Slices, Reducers, Actions, Dispatch, and Selectors

What Redux is

Redux is a central state container. Instead of every screen keeping its own separate copy of important shared data, Redux stores that shared data in one place.

In this app, Redux mainly stores the user's army compositions.

```
Component presses button
-> dispatch(action)
-> Redux store receives action
-> reducer decides how state changes
-> components using that state re-render
```

The store

The Redux store is created in `src/store/index.ts`.

```
const rootReducer = combineReducers({
  armyCompositions: armyCompositionsReducer,
});
```

This means the global Redux state has a property named `armyCompositions`. That property is controlled by the army compositions reducer.

```
{
  armyCompositions: []
}
```

The store is made available to the whole app in `App.tsx`:

```
<Provider store={store}>
  ...
</Provider>
```

Without this provider, components could not use `useAppDispatch` or `useAppSelector`, because React would not know which Redux store to use.

The slice

A slice is Redux Toolkit's way to group one feature's state, reducers, and generated actions.

Your slice is in `src/features/armyCompositions/armyCompositionSlice.ts`.

```
const initialState: ArmyComposition[] = [];
```

This says the army compositions state starts as an empty array.

```
export const armyCompositionSlice = createSlice({
  name: "armyCompositions",
  initialState,
  reducers: {
    armyCompositionAdded(state, action: { payload: ArmyComposition }) {
      state.push(action.payload);
    },
  },
});
```

`createSlice` does several things:

- It gives the feature a name: `armyCompositions`.
- It stores the initial state.
- It defines reducer functions.
- It automatically creates action creator functions with the same names as the reducers.

Reducers

A reducer is a function that changes Redux state based on an action.

Example:

```
armyCompositionAdded(state, action: { payload: ArmyComposition }) {
  state.push(action.payload);
}
```

This means: when Redux receives the `armyCompositionAdded` action, take the new army composition from `action.payload` and add it to the array.

Why `state.push` is allowed

In classic Redux, reducers are not allowed to directly mutate state. But this project uses Redux Toolkit, and Redux Toolkit uses Immer internally.

Immer lets you write code that looks like mutation:

```
state.push(action.payload);
```

But internally, it still creates a safe immutable state update.

Defense answer: Redux Toolkit allows simpler reducer syntax through Immer. Although the code looks like direct mutation, Redux Toolkit safely creates the new immutable state behind the scenes.

Actions and payloads

An action is an object that describes what happened. A payload is the data attached to the action.

This code in your app:

```
dispatch(armyCompositionAdded(newArmyComposition));
```

creates an action similar to this:

```
{
  type: "armyCompositions/armyCompositionAdded",
  payload: newArmyComposition
}
```

The `type` tells Redux what happened. The `payload` contains the new data needed by the reducer.

Dispatch

`dispatch` sends an action to Redux.

```
dispatch(armyCompositionAdded(newArmyComposition));
```

This means: "Hey Redux, an army composition was added. Here is the new army composition. Please update the store."

Redux receives the action, checks the action type, and runs the matching reducer from the slice.

Selectors

A selector reads data from Redux state.

```
export const selectArmyCompositions = (state: RootState) =>
  state.armyCompositions;
```

This selector says: from the full Redux state, return only the army compositions part.

The app reads that data with:

```
const armyCompositions = useAppSelector(selectArmyCompositions);
```

When the Redux state changes, components using this selector automatically re-render with the new data.

Full Redux example from the app

User creates army

- > useCreateArmyComposition creates newArmyComposition
- > dispatch(armyCompositionAdded(newArmyComposition))
- > Redux reducer adds it to the armyCompositions array
- > HomeScreen reads updated Redux state
- > new army card appears
- > Firestore also saves it remotely

Important Redux vocabulary

- **Store:** the central state container.
- **Slice:** one feature's Redux state and reducers.
- **Reducer:** a function that updates state based on an action.
- **Action:** an object describing what happened.
- **Payload:** the data sent with the action.
- **Dispatch:** the function used to send an action to Redux.
- **Selector:** a function used to read data from Redux.

Defense answer: The requirement was to use Redux. In this app, Redux stores army compositions because that data is shared between screens. The slice defines the allowed state changes, dispatch sends actions, reducers update the state, and selectors let screens read the updated data.

9. Redux Toolkit Versus Classic Redux

Classic Redux

Classic Redux usually requires manually writing action type strings, action creator functions, and reducers separately.

```
const ADD_COMPOSITION = "ADD_COMPOSITION";

function addComposition(composition) {
  return {
    type: ADD_COMPOSITION,
    payload: composition,
  };
}
```

That can become repetitive as the app grows.

Redux Toolkit

Redux Toolkit reduces boilerplate. In this app, `createSlice` defines the reducers and automatically generates matching action creators.

```
export const armyCompositionSlice = createSlice({
  name: "armyCompositions",
  initialState,
  reducers: {
    armyCompositionAdded(state, action) {
      state.push(action.payload);
    },
  },
});
```

Redux Toolkit automatically creates:

```
armyCompositionAdded(newArmyComposition)
```

which produces an action with a type similar to:

```
{
  type: "armyCompositions/armyCompositionAdded",
  payload: newArmyComposition
}
```

Why this helps

- Less repetitive code.

- Action creators are generated automatically.
- Reducer logic stays grouped by feature.
- Immer allows simpler immutable update syntax.

Defense answer: The requirement was to use Redux. This app uses Redux Toolkit because it is the modern Redux approach: it groups feature state in slices, generates actions automatically, and reduces boilerplate compared with classic Redux.

10. Immutability and Immer

What immutability means

Immutability means state should not be changed directly. Instead, updates should create a new version of the state. This helps Redux detect changes and update the UI correctly.

The confusing part

This reducer looks like it mutates state:

```
armyCompositionAdded(state, action: { payload: ArmyComposition }) {  
  state.push(action.payload);  
}
```

In classic Redux, directly pushing into state would be wrong. But this app uses Redux Toolkit, and Redux Toolkit uses Immer internally.

What Immer does

Immer lets the reducer code look mutable, but it records the changes and produces a new immutable state behind the scenes.

```
Reducer code:  
state.push(newItem)  
  
Immer internally:  
tracks the change  
creates a new safe state  
Redux receives updated immutable state
```

Example with update

```
if (armyComposition) {  
  armyComposition.units = action.payload.units;  
  armyComposition.totalPoints = action.payload.totalPoints;  
}
```

This also looks like direct mutation. Redux Toolkit allows it because Immer handles the immutable update internally.

Defense answer: Redux state should be immutable. Redux Toolkit uses Immer, so reducers can be written with simple code like `push` or property assignment while still producing safe immutable updates internally.

11. Redux Persist

What Redux Persist does

Redux Persist saves Redux state to local storage and reloads it when the app starts again.

Persist config

```
const persistConfig = {  
  key: "root",  
  storage: AsyncStorage,  
};
```

`key: "root"` names the saved persisted state. `storage: AsyncStorage` tells Redux Persist to store the data locally on the device.

persistReducer

```
const persistedReducer = persistReducer(persistConfig, rootReducer);
```

`persistReducer` wraps the normal root reducer so Redux Persist can save and restore its state.

persistStore

```
export const persistor = persistStore(store);
```

`persistStore` creates the persistor object. The persistor controls the persistence process.

PersistGate

```
<PersistGate loading={null} persistor={persistor}>  
  ...  
</PersistGate>
```

`PersistGate` waits until persisted Redux state has been loaded back into the store before rendering the app UI.

Ignored actions

```
ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER]
```

Redux Persist dispatches internal actions. These are ignored in Redux Toolkit's serializable check to avoid false warnings.

Defense answer: Redux Persist saves Redux state to AsyncStorage and reloads it on app startup. `persistReducer` wraps the reducer, `persistStore` creates the persistor, and `PersistGate` waits for rehydration before rendering the app.

12. Selectors

What a selector is

A selector is a function that reads a specific piece of Redux state.

```
export const selectArmyCompositions = (state: RootState) =>
  state.armyCompositions;
```

This selector receives the full Redux state and returns only the army compositions part.

Using the selector

```
const armyCompositions = useAppSelector(selectArmyCompositions);
```

The component or hook does not need to know the full state structure every time. It uses the selector as the reusable way to read that data.

Why selectors are useful

- They centralize how Redux data is read.
- They make components cleaner.
- They make refactoring easier if the state shape changes later.
- They work well with typed Redux hooks.

Selector and re-rendering

When the selected Redux data changes, components using `useAppSelector` re-render with the new value.

Defense answer: Selectors are reusable functions for reading Redux state. This app uses `selectArmyCompositions` so screens and hooks can read army compositions consistently.

13. Optimistic Updates and Failure Handling

What an optimistic update is

An optimistic update changes the local UI before the remote database operation finishes. It assumes the remote operation will succeed.

Delete example

```
dispatch(armyCompositionDeleted(id));

if (currentUser) {
  await deleteArmyComposition(database, currentUser.uid, id);
}
```

The army composition is removed from Redux first, so the card disappears immediately. Then Firestore deletes the document.

Create example

```
dispatch(armyCompositionAdded(newArmyComposition));

await saveArmyComposition(database, currentUser.uid, newArmyComposition);
```

The app updates local state immediately, then saves the new composition remotely.

Why optimistic updates are used

- The app feels faster.
- The UI responds immediately to the user's action.
- The user does not wait for the network before seeing feedback.

The risk

If Firestore fails after Redux already updated, the UI may temporarily show a state that was not saved remotely.

Production improvement: A stronger version could catch Firestore errors and roll back Redux state, show a retry button, or reload the latest Firestore snapshot.

Why it is still defensible

For this app, optimistic updates are a reasonable pattern because they make the interface responsive. The important defense point is to also know the tradeoff and possible improvement.

Defense answer: The app uses optimistic updates to make the UI feel fast. Redux updates immediately, then Firestore saves or deletes remotely. The tradeoff is that production code should handle failed writes with rollback or retry behavior.

14. How Redux and Firestore Work Together In This App

Firestore is the remote database. Redux is the local app state. React Native screens display the Redux state.

```
Firestore
  -> onSnapshot
  -> useLoadArmyCompositions hook
  -> dispatch
  -> Redux store
  -> selector
  -> screen UI
```

Loading army compositions

The hook `useLoadArmyCompositions` listens to Firestore when a user is logged in.

```
const unsubscribe = subscribeToArmyCompositions(
  database,
  currentUser.uid,
  (armyCompositions) => {
    dispatch(armyCompositionsLoaded(armyCompositions));
  },
);
```

This means Firestore sends the latest army compositions, and the hook dispatches them into Redux.

Creating an army composition

In `useCreateArmyComposition`, the app creates a new object, updates Redux, saves to Firestore, and navigates to the detail screen.

```
dispatch(armyCompositionAdded(newArmyComposition));

await saveArmyComposition(database, currentUser.uid, newArmyComposition);
```

Redux updates immediately. Firestore saves the data remotely.

Deleting an army composition

In `useArmyCompositions`, the app removes the composition locally first, then deletes it from Firestore.

```
dispatch(armyCompositionDeleted(id));

if (currentUser) {
  await deleteArmyComposition(database, currentUser.uid, id);
}
```

This is an optimistic update. The UI changes immediately instead of waiting for the network.

Updating units

In `useArmyCompositionUnits`, one helper updates three places:

- local screen state with `setSelectedUnits`
- Redux state with `dispatch`
- Firestore with `updateArmyCompositionUnits`

```
setSelectedUnits(updatedUnits);

dispatch(
  armyCompositionUnitsUpdated({
    armyCompositionId: armyComposition.id,
    units: updatedUnits,
    totalPoints: updatedTotalPoints,
  }),
);

await updateArmyCompositionUnits(
  database,
  currentUser.uid,
  armyComposition.id,
  updatedUnits,
  updatedTotalPoints,
);
```

Defense answer: Firestore stores the persistent remote data. Redux stores the local app state that screens read from. Hooks connect them by listening to Firestore and dispatching updates into Redux.

15. Short Defense Answers

How does Redux work in this app?

Redux stores army compositions in a central store. The army composition slice defines reducers for loading, adding, updating, clearing, and deleting compositions. Components dispatch actions to request state changes, reducers update the state, and selectors let screens read the updated state.

What is a slice?

A slice is one feature's Redux state and reducer logic grouped together. Redux Toolkit also generates action creators from the reducer names.

What is dispatch?

`dispatch` sends an action to the Redux store. The store then runs the matching reducer to update state.

What is a reducer?

A reducer is a function that receives the current state and an action, then produces the next state.

What is a payload?

The payload is the data sent with an action. For example, when adding an army composition, the payload is the new army composition object.

How does Firestore work in this app?

Firestore stores shared armies in a top-level `armies` collection and user-created army compositions under `users/{userId}/armyCompositions`. The app uses `getDocs` for one-time reads, `onSnapshot` for real-time user data, `setDoc` for saving and updating, and `deleteDoc` for deleting.

How does Firebase Authentication work in this app?

Firebase Authentication handles register, login, logout, and account updates. `AuthContext` listens to Firebase with `onAuthStateChanged`. When Firebase detects a logged-in user, the app stores that user in `currentUser`; when the user logs out, `currentUser` becomes `null`.

Why is auth passed as the first parameter?

Firebase's modular SDK passes the service instance first. Authentication functions receive `auth` first so Firebase knows which Auth instance should handle the operation. Firestore functions similarly use the `database` instance or references created from it.

How does authentication connect to Firestore?

After login, Firebase gives the app a user object with a unique `uid`. The app uses that uid in Firestore paths like `users/{uid}/armyCompositions`, so each user loads and saves their own army compositions.

What design patterns are used around Firestore?

The app uses a service layer, user-owned subcollections, observer/subscription through `onSnapshot`, optimistic UI updates, query-shaped documents, and async data handling with `async / await`.

What is the difference between Firestore and Redux?

Firestore is the remote database. Redux is local app state. Firestore persists and syncs data. Redux makes the data easy and fast for screens to read while the app is running.

What is the auth/account lifecycle?

It is the full identity flow: register, login, restore saved session, reauthenticate for sensitive changes, update email/password, and logout. Firebase Auth owns this lifecycle, while `AuthContext` exposes the current user to the app.

Why does account updating require reauthentication?

Email and password changes are sensitive. Firebase requires recent authentication so the app proves the user still knows their current password before allowing the update.

How is Firestore data modeled?

Shared armies are stored in a top-level `armies` collection. User-created compositions are stored under `users/{uid}/armyCompositions`. Composition documents contain the data the screen needs to render, which matches Firestore's query-shaped data style.

Why are Firestore security rules important?

The frontend builds the correct path with `currentUser.uid`, but backend Firestore rules must enforce that users can only read and write their own documents.

What is the listener lifecycle?

The app starts a Firebase listener with `onSnapshot` or `onAuthStateChanged`, receives updates through a callback, and returns the unsubscribe function from `useEffect` so React can clean it up.

What is Redux Toolkit?

Redux Toolkit is the modern way to write Redux. It uses `createSlice` to group reducers and generate actions, and it uses Immer to make immutable updates easier to write.

What is Immer?

Immer lets reducer code look like it mutates state, such as `state.push(...)`, but internally it creates a safe immutable update for Redux.

What is Redux Persist?

Redux Persist saves Redux state to local storage, here AsyncStorage, and reloads it when the app starts. `PersistGate` waits for that rehydration before rendering the app UI.

What is a selector?

A selector is a reusable function for reading a specific part of Redux state. This app uses `selectArmyCompositions` to read the army compositions list.

What is an optimistic update?

An optimistic update changes Redux/UI before Firestore finishes. It makes the app feel fast, but production code should handle failed remote writes with rollback or retry behavior.

What is the difference between Redux Persist and Firestore persistence?

Redux Persist saves local app state on the device with AsyncStorage. Firestore stores remote account data in the cloud. Redux Persist is local restoration; Firestore is remote persistence and sync.